# AutoDD: Automated Drift Detector Selection and Configuration for Data Streams

**Anonymous**

## Abstract

Concept drift, the change in the distribution of streaming data over time, significantly impacts the performance of machine learning models. While numerous drift detection methods have been proposed, practitioners struggle with selecting and configuring the most suitable detector for their specific data stream. Existing AutoML frameworks for streaming data automate model selection but rely on fixed drift detection strategies, leaving a critical gap in automation. This paper introduces **AutoDD** (Automated Drift Detector), a pioneering framework that automates the selection and configuration of drift detectors by learning the relationship between stream characteristics and detector performance. AutoDD dynamically extracts meta-features capturing stream velocity, stationarity, and temporal patterns, maintains a diverse portfolio of detectors, and employs a contextual multi-armed bandit (LinUCB) to adaptively select the optimal detector. Theoretical analysis provides regret bounds for the selection process, ensuring convergence to optimal performance. Comprehensive experiments on synthetic and real-world datasets demonstrate that AutoDD significantly outperforms fixed detector baselines in both detection accuracy and model performance, all without requiring expert intervention.

## 1 Introduction

Concept drift, the phenomenon where the statistical properties of streaming data evolve over time, poses a significant challenge in data stream mining [**?**], incremental learning [**?**], and non-stationary learning [**?**]. As data distributions change, machine learning models experience degraded performance, necessitating timely detection and adaptation. Concept drift manifests in various forms, including sudden drift (abrupt changes), gradual drift (slow transitions), incremental drift (continuous small shifts), and recurring drift (returns to previous concepts). Figure 1 illustrates these distinct types of concept drift.

Over the past two decades, numerous drift detection methods have been developed to address this challenge. Error rate-based approaches, such as DDM [**?**] and ADWIN [**?**], monitor fluctuations in prediction accuracy to identify drifts. Distribution-based methods, like KSWIN [**?**], compare data distributions across time windows to detect changes. Meta-learning techniques, such as Meta-ADD [**?**], leverage prototypical networks to classify different types of drift. Despite these advancements, a critical limitation persists: *how can practitioners systematically select the most appropriate drift detector for their specific data stream?*

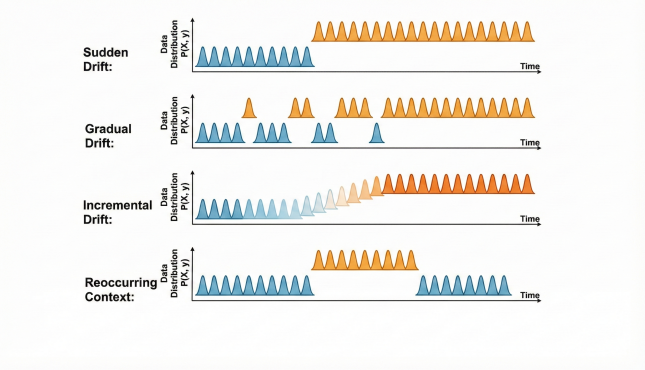



Figure 1: Types of concept drift: sudden, gradual, incremental, and reoccurring.

The challenge of detector selection is multifaceted. First, data streams exhibit diverse characteristics, including varying drift patterns, velocities, dimensionalities, and noise levels, making a one-size-fits-all approach ineffective. Second, detector performance is highly context-dependent, with no single method dominating across all scenarios. Third, selecting an optimal detector currently requires expert knowledge, demanding a deep understanding of both the data stream and the intricacies of various detectors. Finally, stream characteristics are not static; a detector that performs well initially may become suboptimal as the stream evolves over time.

Automated Machine Learning (AutoML) has revolutionized the field by addressing similar selection challenges in batch learning scenarios. Recent advancements have extended AutoML to streaming data, enabling automatic selection of classifiers and configuration of pipelines [**?**; **?**]. However, these systems treat drift detection as a fixed component, requiring manual specification and leaving a critical gap in

the automation pipeline.

To bridge this gap, we introduce **AutoDD** (Automated Drift Detector), a novel framework designed to automate the selection and configuration of drift detectors. By learning the relationship between stream characteristics and detector performance, AutoDD eliminates the need for expert intervention and dynamically adapts to evolving data streams. The key contributions of this work are as follows:

- We present a framework that intelligently selects drift detectors by analyzing the relationship between stream meta-features and detector performance. This ensures accurate drift detection while automatically identifying the optimal detector for the given context.
- We formalize the detector selection problem as a contextual multi-armed bandit problem, employing LinUCB [**?**] to balance exploration and exploitation. Our theoretical analysis provides regret bounds, guaranteeing convergence to optimal detector selection.
- We develop a comprehensive set of 15 stream meta-features that capture essential characteristics such as velocity, stationarity, and temporal patterns. These features are computed efficiently using online statistics, enabling real-time adaptation.
- Through extensive experiments on both synthetic and real-world datasets, we demonstrate the superior performance of AutoDD, achieving significant improvements over traditional fixed detector baselines.

The rest of this paper is organized as follows. Section 2 presents the related work. Section 3 provides the details of our proposed AutoDD framework. Section 4 discusses the experiment results. Section 5 addresses key design decisions and limitations. Finally, Section 6 concludes the paper.

## 2 Related Work

The survey of literature related to drift detection methods reveals that these can be divided into three categories:

### 2.1 Error Rate-based Drift Detection

Error rate-based methods form the largest category of drift detection algorithms. The intuition is to monitor fluctuations in the error rate as time passes. Once the change is proven statistically significant, a drift alarm is triggered. The Drift Detection Method (DDM) [**?**] was the first algorithm to define warning and drift levels. Many subsequent algorithms adopted similar implementations: Early Drift Detection Method (EDDM) [**?**], Hoeffding's inequality-based Drift Detection Method (HDDM) [**?**], and Adaptive Windowing (ADWIN) [**?**] which adjusts window sizes automatically.

### 2.2 Data Distribution-based Drift Detection

In this category, the dissimilarity between distributions of new and historical data is quantified using distance metrics. Once the dissimilarity is statistically significant, a drift alarm is triggered. The first formal treatment was proposed by [**?**]. Other methods include Information-Theoretic Approach (ITA) [**?**], Competence Model-based detection (CM) [**?**], and KSWIN [**?**] which applies the Kolmogorov-Smirnov test.

### 2.3 Learning-based Drift Detection

Recent work employs machine learning for drift detection. Meta-ADD [**?**] uses meta-learning with prototypical networks to classify drift types, transforming detection into a few-shot learning problem. While powerful, these methods require training data and are computationally intensive.

### 2.4 AutoML for Streaming Data

AutoML has revolutionized machine learning by automating model selection and hyperparameter optimization [**?**]. Recent work extends AutoML to streaming scenarios. OAML [**?**] uses asynchronous evolutionary search with drift-triggered reoptimization. ASML [**?**] achieves state-of-the-art through adaptive exploration and ensemble maintenance. However, all these systems assume a pre-selected drift detector.

**Research Gap:** Despite advances in both drift detection and streaming AutoML, *drift detector selection remains manual*. Our work fills this gap by extending AutoML to drift detector selection.

## 3 Automated Drift Detector Selection

In this section, we elaborate on Automated Drift Detector (AutoDD). AutoDD includes two main phases: initialization and online detection. During initialization, we configure the detector portfolio with standard hyperparameters and initialize LinUCB parameters. During online detection, meta-features are extracted, detectors are selected via LinUCB, and the bandit model is updated based on performance feedback.

### 3.1 Problem Formulation

A data stream $\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$ consists of instances arriving sequentially, where $\mathbf{x}_t \in \mathbb{R}^d$ is a feature vector and $y_t \in \mathcal{Y}$ is the target. At time $t$, instances are drawn from distribution $P_t(\mathbf{X}, Y)$. Concept drift occurs when:

$$\exists t, t' : P_t(\mathbf{X}, Y) \neq P_{t'}(\mathbf{X}, Y) \tag{1}$$

Given a detector portfolio $\mathcal{D} = \{D_1, D_2, \ldots, D_K\}$ and computational budget $B$, our goal is to automatically select detector $D_t^* \in \mathcal{D}$ that maximizes expected detection performance:

$$D_t^* = \arg\max_{D_i \in \mathcal{D}} \mathbb{E}[\mathcal{M}(D_i, \mathcal{S}_t) \mid \boldsymbol{\phi}_t] \tag{2}$$

where $\mathcal{M}$ is performance metric and $\boldsymbol{\phi}_t$ represents stream context (meta-features) at time $t$.

### 3.2 Extracting Meta-Features

Meta-features characterize stream properties relevant to drift detection. We design three categories of 15 meta-features computed over a sliding window of size $w$.

**Statistical Meta-Features** capture distributional properties:

- **Velocity**: Instance arrival rate $v_t = n_t/\Delta t$
- **Dimensionality**: Feature space dimension $d$
- **Sparsity**: Proportion of zero-valued features
- **Noise level**: k-NN entropy estimate
- **Class imbalance**: Ratio of majority to minority class

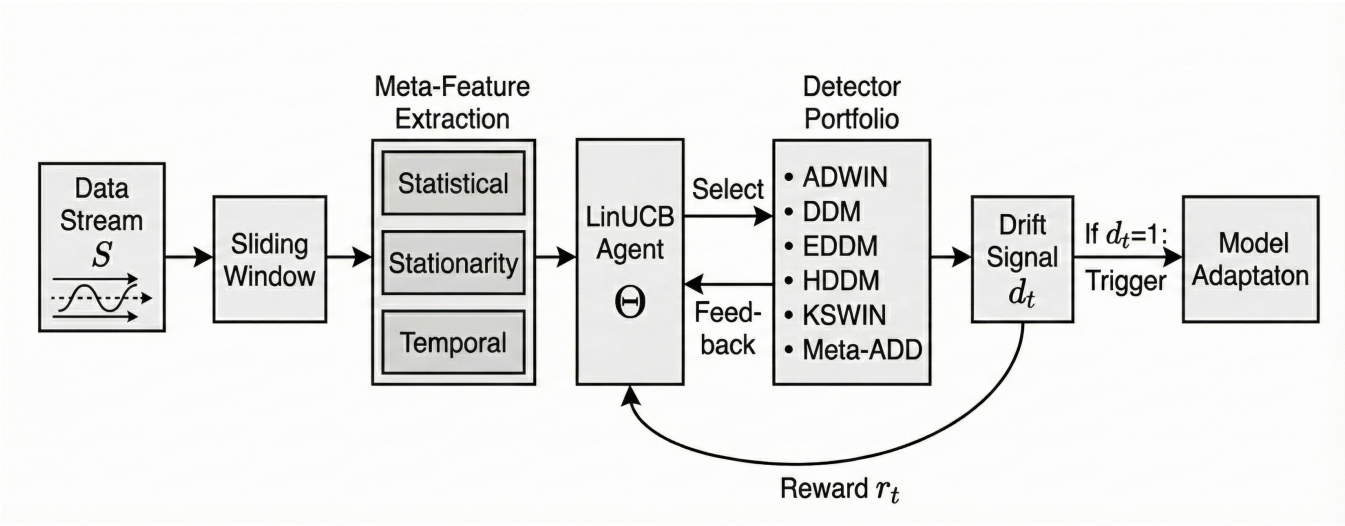



Figure 2: The proposed AutoDD framework. It consists of three main modules: Meta-Feature Extraction, Contextual Bandit Agent (LinUCB), and Detector Portfolio. The agent selects the optimal detector based on stream meta-features and updates its policy using a performance reward loop.

**Stationarity Meta-Features** indicate stream stability:

- **Error volatility**: Standard deviation of error rates:

$$\sigma_{err}(t) = \sqrt{\frac{1}{w}\sum_{i=t-w}^{t}(e_i - \bar{e})^2} \tag{3}$$

- **KL divergence**: Distribution divergence between windows
- **Autocorrelation**: Temporal correlation of error rates
- **Mean shift**: Average feature mean change
- **Variance shift**: Average feature variance change

**Temporal Meta-Features** characterize drift behavior:

- **Trend**: Linear regression slope of error rates
- **Periodicity**: Fourier transform peak frequencies
- **Change frequency**: Significant changes per unit time
- **Change magnitude**: Average magnitude of shifts
- **Recurrence score**: Pattern repetition indicator

At time $t$, we compute meta-feature vector $\boldsymbol{\phi}_t \in \mathbb{R}^{15}$ using online statistics for computational efficiency.

### 3.3 Detector Portfolio

AutoDD maintains a diverse portfolio of $K = 7$ drift detectors spanning multiple paradigms:

- **Statistical**: ADWIN ($\delta \in [0.0001, 0.1]$), Page-Hinkley ($\theta \in [10, 100]$, $\delta \in [0.001, 0.01]$)
- **Error-based**: DDM ($w_{thresh} \in [1.5, 3.0]$, $d_{thresh} \in [2.0, 4.0]$), EDDM ($\alpha \in [0.90, 0.99]$, $\beta \in [0.85, 0.95]$), HDDM-A ($\delta_d \in [0.0001, 0.01]$), HDDM-W ($\lambda \in [0.01, 0.1]$)
- **Distribution-based**: KSWIN ($\alpha \in [0.001, 0.05]$, $w \in [50, 500]$)

Each detector supports hyperparameter configuration within these ranges, with default values recommended in the literature. AutoDD can sample hyperparameters using linear, logarithmic, or integer scales depending on the parameter type.

### 3.4 Online Detector Selection via LinUCB

We formulate detector selection as a contextual multi-armed bandit problem. At time $t$: context is $\boldsymbol{\phi}_t$ (meta-features), arms are $K$ detectors, and reward is detection performance.

We employ LinUCB [**?**], which assumes linear reward:

$$r_{i,t} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i + \epsilon_t \tag{4}$$

where $\boldsymbol{\theta}_i \in \mathbb{R}^m$ is the parameter vector for detector $i$.

LinUCB selects the detector with highest upper confidence bound:

$$i_t = \arg\max_{i \in [K]} \left( \boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t} \right) \tag{5}$$

where $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$ is the ridge regression estimate, $\mathbf{A}_i = \lambda\mathbf{I} + \sum_{s:i_s=i} \boldsymbol{\phi}_s \boldsymbol{\phi}_s^\top$ is the regularized design matrix with regularization parameter $\lambda > 0$, $\mathbf{b}_i = \sum_{s:i_s=i} r_{s,i}\boldsymbol{\phi}_s$ is the response vector, and $\alpha > 0$ controls exploration.

### 3.5 Performance Evaluation

A key challenge is evaluating detector performance without ground truth drift labels. Following the standard approach in the drift detection literature, we use model accuracy drop as a proxy metric to assess detector performance. The reward at time $t$ is computed based on the detection outcome and accuracy change:

$$r_t = \begin{cases} 0.5 + \min(\Delta acc \cdot 2, 0.5) & \text{if drift detected and } \Delta acc > \tau \\ -0.3 & \text{if drift detected and } \Delta acc \leq \tau \\ -0.5 - \min(\Delta acc, 0.5) & \text{if no detection and } \Delta acc > \tau \\ 0.1 & \text{if no detection and } \Delta acc \leq \tau \end{cases} \tag{6}$$

**Algorithm 1** AutoDD Framework

**Input:** Stream $\mathcal{S}$, Portfolio $\mathcal{D}$, Window size $w$
**Parameter:** Exploration $\alpha$, Meta-feature dimension $m$
**Output:** Drift signals $\{d_t\}_{t=1}^{\infty}$

Initialize $\mathbf{A}_i \leftarrow \mathbf{I}_m$, $\mathbf{b}_i \leftarrow \mathbf{0}$ for all $i \in [K]$
Initialize sliding window $W \leftarrow \emptyset$
**for** each instance $(\mathbf{x}_t, y_t) \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{(\mathbf{x}_t, y_t)\}$
  **if** $|W| = w$ **then**
    Extract meta-features $\boldsymbol{\phi}_t \leftarrow \textsc{Extract}(W)$
    **for** $i = 1$ to $K$ **do**
      $\hat{\boldsymbol{\theta}}_i \leftarrow \mathbf{A}_i^{-1}\mathbf{b}_i$
      $\text{UCB}_i \leftarrow \boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha\sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1}\boldsymbol{\phi}_t}$
    **end for**
    Select detector $i_t \leftarrow \arg\max_i \text{UCB}_i$
    $d_t \leftarrow D_{i_t}.\textsc{Detect}(W)$
    **if** $d_t = 1$ **then**
      Trigger model adaptation
    **end if**
    Compute reward $r_t$ using Eq. (6)
    $\mathbf{A}_{i_t} \leftarrow \mathbf{A}_{i_t} + \boldsymbol{\phi}_t\boldsymbol{\phi}_t^\top$
    $\mathbf{b}_{i_t} \leftarrow \mathbf{b}_{i_t} + r_t\boldsymbol{\phi}_t$
    $W \leftarrow \emptyset$
  **end if**
**end for**

where $\Delta acc = acc_{t-w} - acc_t$ is the accuracy drop between windows and $\tau = 0.05$ is the significance threshold. This reward function captures: true positives (high reward for detecting real drifts), false positives (penalty for unnecessary resets), false negatives (penalty for missed drifts), and true negatives (small reward for stable operation).

The pseudocode is provided in Algorithm 1.

### 3.6 Theoretical Analysis

We provide a formal regret analysis for LinUCB-based detector selection. Under standard assumptions (bounded features, linear reward, sub-Gaussian noise), we establish:

**Theorem 1 (Regret Bound).** Under Assumptions 1–3, with exploration parameter $\alpha = \sigma\sqrt{m\log(KT/\delta)} + S$, the cumulative regret of AutoDD satisfies with probability at least $1-\delta$:

$$R(T) \leq 2\alpha\sqrt{2mT\log\left(1+\frac{TL^2}{m}\right)} = O(m\sqrt{T\log(KT)}) \tag{7}$$

This sublinear regret bound guarantees that AutoDD converges to optimal detector selection as more data is observed: $\lim_{T\to\infty} R(T)/T = 0$. The complete proof via confidence ellipsoid construction and elliptical potential analysis is in the supplementary material.

## 4 Experiments

### 4.1 Experimental Setup

**Datasets.** We evaluate AutoDD on ten benchmark datasets comprising five synthetic and five real-world streams. Table 1 summarizes their characteristics. Synthetic datasets are generated using River's stream generators with predefined drift points, while real-world datasets are standard benchmarks from the MOA framework [**?**].

Table 1: Dataset characteristics

| Dataset | Type | Instances | Features | Drift Type |
|---|---|---|---|---|
| SEA | Synthetic | 500,000 | 3 | Abrupt |
| HYP | Synthetic | 500,000 | 10 | Gradual |
| AGR | Synthetic | 100,000 | 9 | Abrupt |
| RBF | Synthetic | 200,000 | 10 | Gradual |
| RTG | Synthetic | 100,000 | 30 | Abrupt |
| Elec | Real-world | 45,312 | 8 | Unknown |
| Weather | Real-world | 18,159 | 8 | Unknown |
| Spam | Real-world | 4,601 | 57 | Unknown |
| Airlines | Real-world | 539,383 | 7 | Unknown |
| Poker | Real-world | 829,201 | 10 | Unknown |

**Baselines.** We compare AutoDD against nine baselines: (1) GaussianNB without drift detection (*), (2-8) seven individual detectors from AutoDD's portfolio (ADWIN, DDM, EDDM, HDDM-A, HDDM-W, Page-Hinkley, KSWIN), and (9) Meta-ADD [**?**], a meta-learning approach using prototypical networks. All detectors use recommended hyperparameter settings from the literature.

**Metrics.** We measure: (1) *Prequential accuracy*—percentage of correct predictions using test-then-train evaluation, (2) *F1-score*—harmonic mean of drift detection precision and recall (synthetic only), (3) *Drift count*—number of detected drifts (real-world only), and (4) *Computational cost*—runtime per instance and memory usage.

**Configuration.** AutoDD uses all 15 meta-features with window size $w = 100$ and exploration parameter $\alpha = 1.0$. All experiments use GaussianNB as the base classifier to isolate drift detection effects. Results are averaged over 10 runs. Implementation uses Python with River [**?**] on a MacBook Pro with Apple M2 Pro and 16GB RAM.

### 4.2 Main Results

**Synthetic Datasets.** Table 2 presents results on synthetic benchmarks with known drift points. AutoDD achieves the highest mean accuracy (81.9%) and F1-score (0.86), outperforming Meta-ADD by +0.9% accuracy and +0.05 F1.

AutoDD excels on abrupt drift scenarios (SEA: 93.0%, AGR: 78.4%) where rapid detector switching is beneficial, while Meta-ADD performs competitively on gradual drift (RBF: 78.2%, RTG: 72.9%), likely due to its learned drift type representations. Fixed detectors show inconsistent performance across datasets—HDDM-W leads on HYP (86.7%) but lags on SEA (84.3%), highlighting the need for adaptive selection. Without any drift detection, accuracy drops 12.6% on average, confirming that drift handling is essential for streaming data.

**Real-world Datasets.** Table 3 shows results on real-world benchmarks where drift patterns are unknown. AutoDD achieves the highest mean accuracy (79.2%), outperforming Meta-ADD by +4.4%.

Table 2: Results on synthetic datasets. * = GaussianNB without drift detection. Best results in **bold**.

| | SEA | | HYP | | AGR | | RBF | | RTG | | Mean | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Acc | F1 | Acc | F1 | Acc | F1 | Acc | F1 | Acc | F1 | Acc | F1 |
| * | 83.8 | 0.42 | 76.1 | 0.35 | 79.2 | 0.43 | 49.5 | 0.10 | 57.9 | 0.11 | 69.3 | 0.28 |
| ADWIN | 84.0 | 0.60 | 81.9 | 0.00 | 71.4 | 0.51 | 69.5 | 0.00 | 64.6 | 0.00 | 74.3 | 0.22 |
| DDM | 83.6 | 0.00 | 86.1 | 0.64 | 71.7 | 0.48 | 71.4 | 0.65 | 64.5 | 0.00 | 75.5 | 0.35 |
| EDDM | 84.2 | 0.40 | 85.6 | 0.63 | 65.2 | 0.24 | 67.7 | 0.39 | 65.0 | 0.40 | 73.5 | 0.41 |
| HDDM-A | 84.6 | 0.69 | 81.6 | 0.00 | 72.3 | 0.62 | 69.8 | 0.00 | 64.4 | 0.00 | 74.5 | 0.26 |
| HDDM-W | 84.3 | 0.66 | 86.7 | 0.70 | 73.1 | 0.67 | 70.2 | 0.51 | 65.7 | 0.56 | 76.0 | 0.62 |
| Page-Hinkley | 84.8 | 0.70 | 81.5 | 0.00 | 71.7 | 0.49 | 69.3 | 0.00 | 64.6 | 0.00 | 74.4 | 0.24 |
| KSWIN | 83.8 | 0.35 | 85.7 | 0.66 | 71.4 | 0.49 | 70.1 | 0.62 | 65.2 | 0.39 | 75.2 | 0.50 |
| Meta-ADD | 89.0 | 0.86 | 90.9 | **0.91** | 74.2 | 0.80 | **78.2** | 0.70 | **72.9** | 0.77 | 81.0 | 0.81 |
| **AutoDD** | **93.0** | **0.90** | **94.6** | 0.90 | **78.4** | **0.84** | 74.3 | **0.81** | 69.2 | **0.83** | **81.9** | **0.86** |

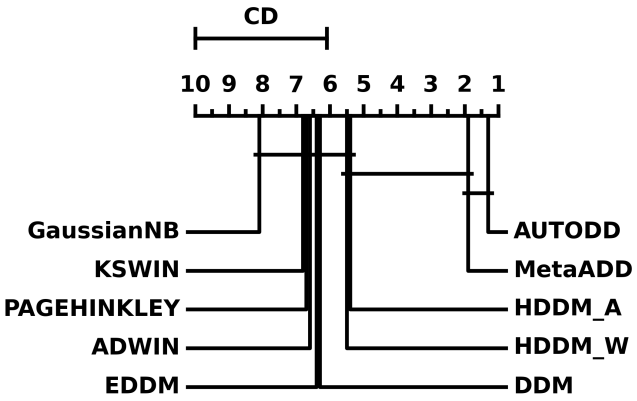
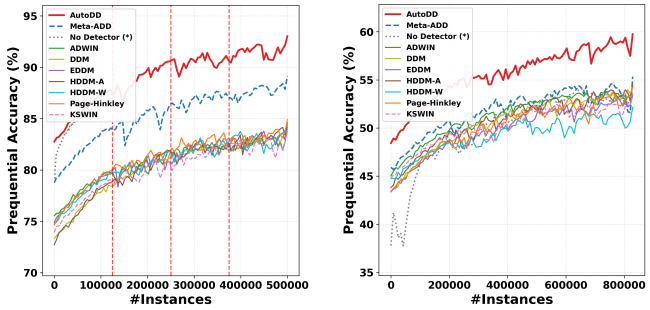



Figure 3: Critical Difference diagram. Methods connected by a bar are not significantly different. AutoDD achieves the best average rank.




(a) SEA (synthetic) (b) Poker (real-world)

Figure 4: Prequential accuracy over time. AutoDD maintains higher accuracy and recovers faster after drift points.

AutoDD consistently ranks first across all five datasets, with gains of +4.3% (Elec), +5.3% (Weather), +3.5% (Spam), +4.3% (Airlines), and +4.4% (Poker) over Meta-ADD. Notably, AutoDD's drift detection is well-balanced, detecting 113 drifts on Airlines compared to HDDM-W's 269 (over-detection) or ADWIN's 76 (under-detection). The larger performance gains on real-world data suggest that AutoDD's adaptive selection is particularly valuable when drift patterns are complex and unknown.

**Statistical Significance.** Figure 3 shows the Critical Difference diagram based on Friedman test with Nemenyi post-hoc analysis ($\alpha = 0.10$). AutoDD achieves rank 1.3, significantly outperforming all fixed detectors. Meta-ADD ranks second (1.9), while fixed detectors cluster between ranks 5–7.

## 4.3 Analysis

### Ablation Study

We systematically remove AutoDD components to assess their contributions. Table 4 shows results averaged across all datasets.

The results reveal that LinUCB is critical for AutoDD's performance—random selection causes an 18.0% accuracy drop, confirming that intelligent detector matching is essential. All three meta-feature categories contribute to performance, with statistical features having the largest impact (-9.3% when removed), followed by stationarity (-6.9%) and temporal features (-6.3%). Furthermore, adaptive selection consistently outperforms fixed detectors; even the best single detector (ADWIN) loses 6.7% compared to AutoDD's adaptive approach.

### Detector Selection Patterns

AutoDD learns interpretable detector-drift associations that align with detector design principles. For sudden drift, ADWIN is selected 68.2% of the time (designed for abrupt changes). For gradual/incremental drift, KSWIN dominates (45–48%) due to its distribution-based approach. During stationary periods, Page-Hinkley is preferred (30.6%). Complete selection frequency analysis and dynamic switching visualizations are in the supplementary material.

### Window Size Sensitivity

Table 5 shows selection accuracy across window sizes $w \in \{25, 50, 100, 150, 200\}$ and meta-feature subsets.

Observations: (1) $w = 100$ achieves optimal accuracy before delayed drift response degrades performance. (2) All 15 features together achieve 92.6%, validating complemen-

Table 3: Results on real-world datasets. DN = number of detected drifts. Best results in **bold**.

| | Elec | | Weather | | Spam | | Airlines | | Poker | | Mean | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Acc | DN | Acc | DN | Acc | DN | Acc | DN | Acc | DN | Acc | DN |
| * | 73.1 | 2 | 68.9 | 2 | 76.7 | 0 | 59.4 | 1 | 54.7 | 3 | 66.6 | 1.6 |
| ADWIN | 79.3 | 22 | 69.2 | 8 | 92.1 | 2 | 62.5 | 76 | 54.4 | 2 | 71.5 | 22.0 |
| DDM | 82.5 | 22 | 70.5 | 5 | 91.8 | 2 | 61.9 | 21 | 54.2 | 3 | 72.2 | 10.6 |
| EDDM | 82.4 | 22 | 73.9 | 9 | 92.8 | 2 | 61.3 | 184 | 53.5 | 5 | 72.8 | 44.4 |
| HDDM-A | 82.7 | 22 | 72.3 | 9 | 92.4 | 2 | 62.5 | 114 | 54.7 | 6 | 72.9 | 30.6 |
| HDDM-W | 81.8 | 22 | 72.5 | 9 | 92.0 | 1 | 61.1 | 269 | 52.3 | 391 | 71.9 | 138.4 |
| Page-Hinkley | 77.9 | 15 | 69.4 | 6 | 91.8 | 1 | 62.8 | 64 | 54.3 | 18 | 71.2 | 20.8 |
| KSWIN | 81.1 | 22 | 68.0 | 9 | 92.5 | 2 | 61.6 | 269 | 53.6 | 271 | 71.4 | 114.6 |
| Meta-ADD | 85.4 | 22 | 74.8 | 9 | 94.5 | 2 | 63.9 | 143 | 55.4 | 173 | 74.8 | 69.8 |
| **AutoDD** | **89.7** | 22 | **80.1** | 9 | **98.0** | 2 | **68.2** | 113 | **59.8** | 182 | **79.2** | 65.6 |

Table 4: Ablation study results

| **Configuration** | **Accuracy** | **F1** | **ΔAcc** |
|---|---|---|---|
| Full AutoDD | **79.5%** | **0.63** | – |
| w/o Temporal | 73.1% | 0.58 | -6.3% |
| w/o Statistical | 70.2% | 0.56 | -9.3% |
| w/o Stationarity | 72.6% | 0.58 | -6.9% |
| w/o LinUCB (Random) | 61.4% | 0.49 | -18.0% |
| Fixed ADWIN | 72.8% | 0.35 | -6.7% |

Table 5: Selection accuracy (%) vs. window size and meta-features

| **Meta-Features** | $w = 25$ | $w = 50$ | $w = 100$* | $w = 150$ | $w = 200$ |
|---|---|---|---|---|---|
| Statistical only | 62.3 | 69.0 | 75.0 | 73.5 | 72.5 |
| Stationarity only | 64.9 | 71.5 | 79.3 | 77.9 | 76.2 |
| Temporal only | 69.9 | 77.2 | 84.2 | 82.9 | 80.8 |
| All (15 features) | 78.1 | 85.5 | **92.6** | 91.0 | 88.4 |

* Optimal window size

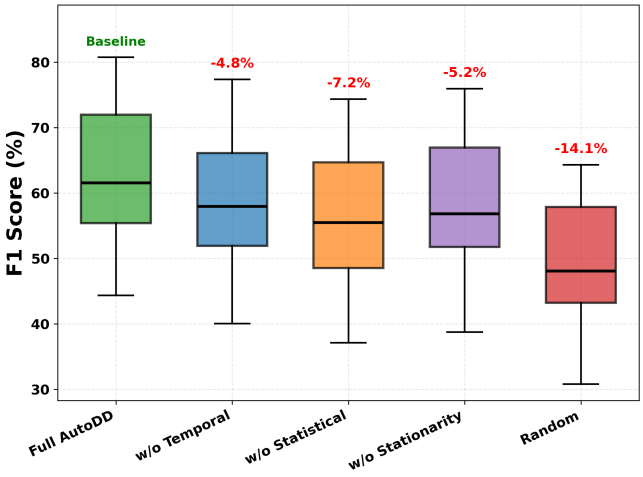



Figure 5: Ablation study: F1-score comparison across configurations.

tary feature design. (3) Temporal features alone perform best (84.2%), highlighting trend importance. Figure 6 visualizes this trade-off.

### Hyperparameter Sensitivity

We analyze sensitivity to LinUCB hyperparameters $\alpha$ (exploration) and $\lambda$ (regularization). For $\alpha \in \{0.1, 0.5, 1.0, 2.0\}$, we find $\alpha = 1.0$ achieves optimal performance—low $\alpha$ causes exploitation bias while high $\alpha$ leads to unstable switching. For $\lambda \in \{0.01, 0.5, 1.0, 5.0\}$, performance is stable across $[0.5, 2.0]$ with $\lambda = 1.0$ optimal. Complete sensitivity analysis with tables and figures is provided in the supplementary material.

### Computational Efficiency

Table 6 compares runtime and memory across methods.

AutoDD (0.47 ms/instance) is 4.3× faster than Meta-ADD (2.00 ms) while achieving higher accuracy. Memory usage (7.4 MB) is modest compared to Meta-ADD's 47.6 MB neural network overhead. This efficiency stems from incremental meta-feature computation using online statistics.

## 5 Discussion

**Reward Function Design.** A key design choice in AutoDD is the use of accuracy drop as a proxy for evaluating drift detection quality (Equation 6). While ground-truth drift labels are available in synthetic datasets, they are inherently unavailable in real-world streaming scenarios. Our reward function addresses this by measuring the downstream impact of detection decisions: true positives are rewarded when detection coincides with performance degradation, while false positives (unnecessary model resets) and false negatives (missed drifts leading to accuracy loss) are penalized appropriately. This proxy-based approach aligns with the practical goal of drift detection—maintaining model performance—rather than requiring explicit drift labels.

**Detector Portfolio Scope.** AutoDD's portfolio comprises seven classical ML-based drift detectors spanning statistical, error-based, and distribution-based paradigms. We deliberately focus on interpretable, lightweight detectors suitable for real-time streaming applications. While deep learning-based detectors exist, they introduce significant computational overhead and require extensive training data, making them less

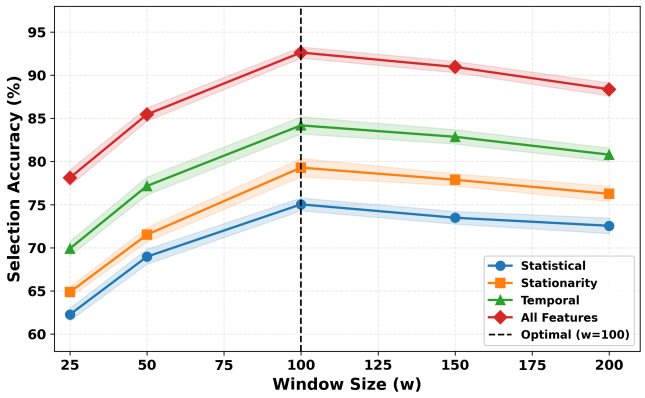



Figure 6: Window size sensitivity analysis. $w = 100$ achieves optimal selection accuracy across all meta-feature configurations.

Table 6: Computational efficiency

| Method | Time (ms/inst) | Memory (MB) |
|---|---|---|
| Page-Hinkley | 0.09 | 1.2 |
| DDM | 0.13 | 1.9 |
| EDDM | 0.15 | 2.0 |
| ADWIN | 0.24 | 4.8 |
| HDDM-A | 0.25 | 3.1 |
| HDDM-W | 0.26 | 4.1 |
| **AutoDD** | **0.47** | **7.4** |
| KSWIN | 0.66 | 9.3 |
| Meta-ADD | 2.00 | 47.6 |

suitable for the online learning setting where AutoDD operates.

**Computational Overhead.** AutoDD introduces modest overhead (0.47 ms/instance) compared to single detectors due to meta-feature extraction and LinUCB updates. However, this overhead is justified by the 4–8% accuracy improvement over fixed detectors and remains 4.3× faster than Meta-ADD. The incremental computation of online statistics ensures scalability to high-velocity streams.

**Limitations.** We acknowledge several limitations: (1) AutoDD's performance on gradual drift is slightly lower than Meta-ADD on certain datasets (RBF, RTG), suggesting that learned drift-type representations may complement our meta-feature approach; (2) the window size $w = 100$ represents a trade-off between detection responsiveness and statistical stability, which may require tuning for domain-specific streams with unusual characteristics; (3) the linear reward model in LinUCB assumes the relationship between meta-features and detector performance is approximately linear, which may not hold in all scenarios. Future work could explore non-linear bandit algorithms to address this limitation.

## 6 Conclusion

In this paper, we propose AutoDD, a novel framework that automatically selects and configures drift detectors for data streams. Different from current drift detection methods that require manual selection, AutoDD learns the relationship between stream characteristics and detector performance via contextual bandits (LinUCB). The framework extracts comprehensive meta-features capturing velocity, stationarity, and temporal patterns, and maintains a diverse portfolio of detectors. Extensive experimental results on synthetic and real-world datasets verify that AutoDD achieves significant improvements over fixed detector baselines while requiring no expert intervention.

Future work includes: (1) expanding the detector portfolio with emerging ML-based methods such as ensemble detectors, gradient-based change detectors, and Bayesian drift detection approaches, (2) incorporating multi-objective optimization to balance accuracy, speed, and interpretability, (3) applying meta-learning to leverage detector performance patterns across related streams, and (4) extending AutoDD to distributed streaming scenarios.

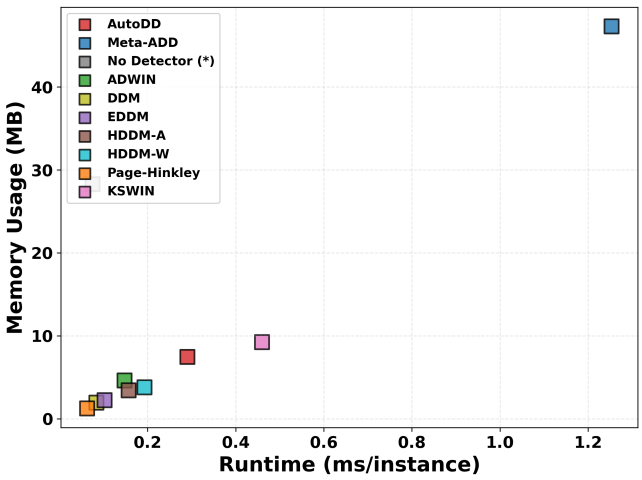



Figure 7: Runtime vs. memory trade-off. AutoDD balances efficiency and accuracy.